DBMS -- ACID Properties & Relational Model Concepts

Target: Explain all 4 ACID properties in 40s with one real example each.
Also explain Degree vs Cardinality -- commonly confused.

ACID Properties
ACID = the 4 guarantees a reliable database must give during transaction processing.

A transaction = a single unit of work (e.g., bank transfer: debit one account, credit another).
Both must succeed together -- or neither should happen.

A -- Atomicity
"All or Nothing."
A transaction is treated as a single unit. Either all operations succeed or none of them are applied.
Example:
```
Transfer Rs.5000 from Account A to Account B:
  Step 1: Debit  Rs.5000 from A   [OK]
  Step 2: Credit Rs.5000 to B       (system crash)
Without Atomicity   A loses Rs.5000, B gets nothing. Data is corrupted.
With Atomicity      Both steps rollback. Neither change is saved.
```
Key point: If any step in a transaction fails, the entire transaction is rolled back.

C -- Consistency
"Valid state to valid state."
After a transaction completes (success or failure), the database must remain in a consistent, valid state -- all rules, constraints, and integrity checks must still hold.
Example:
```
Rule: Account balance cannot go below Rs.0.
Transaction: Withdraw Rs.10,000 from account with Rs.2,000 balance.
Without Consistency   Balance goes to -Rs.8,000. Rule violated.
With Consistency      Transaction is rejected. Balance stays at Rs.2,000.
```
Key point: Constraints (PK, FK, CHECK) + business rules must hold before AND after every transaction.

I -- Isolation
"Concurrent transactions don't interfere with each other."
Even if multiple transactions run at the same time, each transaction executes as if it's running alone. Intermediate states are not visible to other transactions.
Example:
```
Transaction T1: Read balance (Rs.5000)   Deduct Rs.1000   Write Rs.4000
Transaction T2: Read balance (Rs.5000)   Deduct Rs.2000   Write Rs.3000
Without Isolation   T2 reads stale data; final balance = Rs.3000 (T1's change lost)
With Isolation      Transactions are serialized; final balance = Rs.2000 (both applied correctly)
```
Isolation levels (lowest to strictest):
Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	[OK] Possible	[OK] Possible	[OK] Possible
Read Committed	Prevented	[OK] Possible	[OK] Possible
Repeatable Read	Prevented	Prevented	[OK] Possible
Serializable	Prevented	Prevented	Prevented

For PSU interview: just know the 4 levels exist; Serializable is strictest.

Isolation Levels -- Deep Dive (The 3 Anomalies)

Context: In backend systems like MERN APIs, multiple users hit the DB simultaneously.
Isolation levels = rules the DB follows to prevent concurrent transactions from corrupting data.
The trade-off is always: Data Accuracy vs System Performance.

The 3 Read Anomalies (Problems Isolation Solves)
1. Dirty Read

T1 updates a row but hasn't committed yet. T2 reads that uncommitted data.
If T1 rolls back -- T2 worked with data that technically never existed.

T1: UPDATE balance = 4000   (not committed yet)

T2: READ balance sees 4000 DIRTY READ
T1: ROLLBACK balance goes back to 5000
T2 made decisions based on 4000 that never officially existed
2. Non-Repeatable Read

T1 reads a row. Before T1 finishes, T2 updates that same row and commits.
T1 reads the row again -- the data has changed mid-transaction.

T1: READ balance 5000
T2: UPDATE balance = 3000, COMMIT
T1: READ balance again 3000 value changed! Non-Repeatable Read
3. Phantom Read

T1 runs a query to find a set of rows (e.g., "all transactions > Rs.1000").
Before T1 finishes, T2 inserts a new row matching that condition and commits.
T1 runs the same query again -- a "phantom" row magically appears.

T1: SELECT * WHERE amount > 1000 returns 5 rows
T2: INSERT a new row with amount = 2000, COMMIT
T1: SELECT * WHERE amount > 1000 returns 6 rows phantom row appeared!

What Each Isolation Level Does
Read Uncommitted (Lowest)
- Almost no locking. Transactions can read uncommitted draft changes.
- Very fast, highly unsafe.
- Dirty Reads are possible.
Read Committed
- A transaction can only read fully committed data.
- Fixes Dirty Reads. But another transaction can still update the row while you're mid-transaction.
- Default level in most DBs (PostgreSQL, Oracle, SQL Server).
Repeatable Read
- When you read a row, the DB locks it. No other transaction can update/delete that row until you finish.
- If you read it again, guaranteed same value.
- Prevents Dirty + Non-Repeatable Reads. Phantom Reads still possible.
- Default in MySQL/InnoDB.
Serializable (Strictest)
- Locks everything needed -- including ranges of tables to prevent new row insertions.
- Forces concurrent transactions to run as if sequential (one after another).
- Prevents all 3 anomalies. Slowest -- transactions queue up.

Isolation Level Summary -- One Line Each
```
Read Uncommitted  reads draft data  fastest, most dangerous
Read Committed  reads committed only  fixes dirty read
Repeatable Read  locks rows you read  fixes dirty + non-repeatable
Serializable  locks everything  fixes all anomalies, slowest
PSU Interview key point: Serializable is the STRICTEST isolation level.
```

D -- Durability
"Once committed, always committed."
After a transaction is successfully committed, its changes persist permanently -- even if the system crashes, power goes out, or the server restarts.
Example:
```
You book a train ticket  payment confirmed  COMMIT issued.
Server crashes 1 second later.
Without Durability  Ticket booking is lost. Money debited but no ticket.
With Durability  Booking is recovered from disk on restart. Ticket exists.
```
How it's achieved: Write-ahead logs (WAL), journaling, disk flushes after COMMIT.

ACID -- Summary Table

Property
Guarantee
Failure Scenario Prevented

Atomicity
All or nothing

Partial transaction leaving data in broken state

Consistency
Valid state valid state
Constraint violations after transaction

Isolation
No interference between concurrent txns
Dirty reads, lost updates, race conditions

Durability
Committed data survives crashes
Data loss after system failure post-commit

Your 40-Second Script -- ACID

"ACID stands for Atomicity, Consistency, Isolation, and Durability.
Atomicity means a transaction is all-or-nothing -- if any step fails, everything rolls back.
Consistency means the database moves from one valid state to another -- all constraints hold.
Isolation means concurrent transactions don't interfere -- each runs as if it's alone.
Durability means once committed, the data persists even through system crashes.
Classic example: a bank transfer -- debit and credit must both succeed, or neither should."

Relational Model Concepts
The relational model organizes data into tables (relations). Here are the key terms you must know.

1. Attribute
An attribute is a column in a table -- it represents a property of an entity.
```
students table:  Roll_No | Name | Age | Subject
        These are all ATTRIBUTES (columns)
```

2. Tuple
A tuple is a single row in a table -- it represents one record.
students table:

Roll_No	Name	Age	
1	Avanish	22	This entire row = one TUPLE
2	Rahul	21	Another TUPLE

3. Relation (Table)
A relation is the table itself -- a set of tuples that share the same attributes.

In the Relational Model, the formal word for "table" is relation.

4. Relation Schema
The relation schema = the table's name + list of attributes (the structure, without the data).
```
students (Roll_No, Name, Age, Subject)
Relation name + attribute list = RELATION SCHEMA
```
Think of it as the blueprint -- like a class definition in programming before you create objects.

5. Degree
The degree of a relation = the number of attributes (columns) in the table.
students (Roll_No, Name, Age, Subject)
Degree = 4 (4 columns)

Memory trick: Degree = Dimensions (how wide the table is)

6. Cardinality
The cardinality of a relation = the total number of tuples (rows) in the table.
students table with 50 students enrolled Cardinality = 50

Memory trick: Cardinality = Count of rows (how tall the table is)

7. Relation Key
Every row in a relation has one, two, or multiple attributes that can uniquely identify it -- this is the relation key (what we call PK/CK in practice).

Degree vs Cardinality -- The Confusion Cleared

Term
What it counts
Analogy

Degree
Number of columns (attributes)
Width of the table

Cardinality
Number of rows (tuples)
Height of the table

```
        Col1  Col2  Col3  Col4
                        Degree = 4 (4 columns)
Row 1       A     B     C     D
Row 2       E     F     G     H
Row 3       I     J     K     L
                    Cardinality = 3 (3 rows)
```

Don't confuse with cardinality in ER Model (1:1, 1:N, M:N) -- that's a different use of the same word. ER cardinality = "how many entities participate in a relationship." Relational cardinality = "number of rows in a table."

All Relational Model Terms -- Quick Reference
Relation = Table
Tuple = Row (single record)
Attribute = Column (property)
Relation Schema = Table name + column list (the structure)
Degree = Number of columns
Cardinality = Number of rows
Relation Key = Attribute(s) that uniquely identify a tuple (PK/CK)
Domain = Set of allowed values for an attribute (e.g., Age: 0-150)

Follow-Up Questions (Expect These)
Q: What is a domain in the relational model?

The set of all permitted values for an attribute. For example, the domain of Age might be integers from 0 to 150. The domain of Gender might be {Male, Female, Other}.

Q: What is the difference between a relation schema and a relation instance?

Schema = the structure (column names and types) -- fixed and rarely changes.
Instance = the actual data in the table at a given point in time -- changes with every INSERT/UPDATE/DELETE.

Q: What is a dirty read?

Reading data from a transaction that hasn't been committed yet. If that transaction rolls back, you read data that never officially existed.